
DOMINANDO OS VERBOS HTTP

A Semântica Profunda das APIs REST no Backend

Guia Completo para Engenheiros de Software

Introdução

Bem-vindo a este guia aprofundado. Ao arquitetar integrações e desenvolver microsserviços, um dos pilares fundamentais é o correto emprego do protocolo HTTP. Muitas vezes, desenvolvedores utilizam os verbos de forma indiscriminada (transformando tudo em POST), o que destrói as vantagens inerentes ao modelo arquitetural REST (Representational State Transfer).

Neste material, vamos mergulhar na semântica real de cada método. Uma API bem projetada não apenas facilita a vida de quem a consome (como as aplicações frontend ou scripts de automação), mas também tira proveito de mecanismos da própria infraestrutura da web, como proxies, CDNs e balanceadores de carga.

A semântica HTTP é o vocabulário da sua API. Usar o verbo errado é como usar a palavra errada em uma frase: a comunicação até pode acontecer, mas a intenção se perde e bugs sutis começam a surgir, especialmente em ecossistemas distribuídos.

Conceitos Fundamentais: Segurança e Idempotência

Antes de detalharmos cada verbo, é imperativo compreender dois conceitos definidos pelas especificações do HTTP (RFC 7231): **Segurança (Safety)** e **Idempotência (Idempotency)**.

1. Métodos Seguros (Safe Methods)

Um método é considerado *seguro* se ele não altera o estado do recurso no servidor. Ele é essencialmente de leitura. Isso significa que o cliente pode fazer a requisição várias vezes sem medo de modificar dados ou disparar cobranças indesejadas. Os verbos seguros mais comuns são GET, HEAD e OPTIONS.

2. Idempotência

Um método é *idempotente* se o efeito no servidor de realizar a mesma requisição uma única vez for idêntico ao de realizá-la múltiplas vezes. Isso é vital para a tolerância a falhas. Se uma conexão cair e o

cliente tentar novamente (retry), uma operação idempotente garante que não haverá efeitos colaterais indesejados (como duplicar um registro).

Verbo HTTP	Seguro (Safe)?	Idempotente?
GET	Sim	Sim
POST	Não	Não
PUT	Não	Sim
PATCH	Não	Não (Geralmente)
DELETE	Não	Sim

Explorando os Verbos HTTP

Para ilustrar o funcionamento de cada verbo, utilizaremos um contexto prático e realista: o consumo de uma API para gerenciamento e busca de **contratos públicos** e atas de registro de preços.

GET

Safe

Idempotente

O GET é estritamente usado para recuperar a representação de um recurso. Ele nunca deve ter um "body" (corpo) na requisição, e todos os parâmetros de filtragem devem ser passados via Query String ou como parte da rota da URL.

Exemplo de Requisição (Buscando um contrato específico):

```
GET /api/v1/contratos/10294 HTTP/1.1 Host: api.orgaopublico.gov.br Accept: application/json
```

Como o GET é seguro, os navegadores e CDNs podem e vão fazer cache dessa resposta. Se você usar GET para disparar uma ação de exclusão, correrá o risco de um web crawler acidentalmente deletar seus dados ao indexar a página.

POST

Not Safe

Não Idempotente

O POST é o "coringa", mas sua semântica primária é a criação de um recurso subordinado sob uma coleção. Ao enviar um POST, o cliente não sabe qual será o identificador final do recurso (a URL). O servidor é responsável por processar o payload, criar o recurso e retornar a localização dele.

Exemplo prático em Python (Criando um novo edital de licitação):

```
import requests url = "https://api.orgaopublico.gov.br/api/v1/licitacoes"
payload = { "objeto": "Aquisição de Switches de Borda", "valor_estimado": 450000.00, "modalidade": "Pregão Eletrônico" } response = requests.post(url, json=payload) print(response.status_code) # Idealmente 201 Created
```

Por não ser idempotente, se o cliente enviar este mesmo script duas vezes, serão criados dois processos de licitação diferentes no sistema.

PUT

Not Safe

Idempotente

O PUT é usado para a substituição total de um recurso. O cliente deve conhecer o URI exato do recurso e enviar a sua representação completa no corpo da requisição. Se o recurso não existir, o PUT pode, de acordo com a especificação, criar o recurso naquela URI (embora a prática moderna de APIs prefira retornar erro 404).

Exemplo (Atualizando os dados completos do contrato):

```
PUT /api/v1/contratos/10294 HTTP/1.1 Content-Type: application/json { "id":  
10294, "status": "VIGENTE", "valor_total": 520000.00, "fornecedor_cnpj":  
"12.345.678/0001-90" }
```

O PUT exige idempotência. Fazer este request 1 vez ou 100 vezes terá exatamente o mesmo estado final no banco de dados.

PATCH

Not Safe

Não Idempotente (Geralmente)

Diferente do PUT, o PATCH aplica modificações parciais. Você envia apenas os campos que deseja alterar. Isso economiza banda e previne sobrescritas acidentais em campos que o cliente não tinha o valor mais recente.

Exemplo (Aditivando o valor do contrato):

```
PATCH /api/v1/contratos/10294 HTTP/1.1 Content-Type: application/json {  
"valor_total": 600000.00 }
```

Nota de arquitetura: Embora você possa desenhar um PATCH para ser idempotente (como no exemplo acima), a especificação HTTP não obriga a idempotência para o PATCH, especialmente se a operação for algo como {"operacao": "incrementar", "valor": 50}.

DELETE

Not Safe

Idempotente

Sem surpresas, é utilizado para remover o recurso especificado. O sucesso de um DELETE normalmente retorna um status 204 No Content ou 200 OK (caso retorne um recibo da exclusão).

A idempotência do DELETE às vezes causa confusão. Se você deletar algo, a primeira chamada retorna 200. A segunda pode retornar 404. Mas a definição de idempotência baseia-se no *estado do servidor*. Independentemente de retornar 204 ou 404, o estado final do servidor (o recurso não existir mais) permanece o mesmo, honrando a idempotência.

Boas Práticas de Modelagem RESTful

1. Substantivos no Plural para Coleções

Os verbos HTTP já representam a "ação". Portanto, sua URI deve conter apenas substantivos (recursos). Não misture verbos na URI.

- ❌ **Errado:** POST /api/criarContrato
- ❌ **Errado:** GET /api/buscarContratos?id=123
- ✅ **Correto:** POST /api/contratos (O POST já diz que é criar)
- ✅ **Correto:** GET /api/contratos/123

2. Códigos de Status (Status Codes)

Usar o verbo HTTP correto é o primeiro passo, e responder com o Status Code HTTP adequado é o segundo. Uma API bem modelada retorna:

- 200 OK: Genérico para sucesso (GET, PUT, PATCH).
- 201 Created: Retorno obrigatório quando um POST cria um recurso. Deve conter o cabeçalho Location com a URL do novo recurso.
- 204 No Content: Sucesso, mas sem corpo de resposta (muito usado no DELETE).
- 400 Bad Request: Erro de validação da requisição pelo cliente.
- 404 Not Found: Recurso não localizado (muito comum para GET em IDs inexistentes).

- 405 Method Not Allowed: Quando tenta-se fazer um POST em uma URI que só aceita GET, por exemplo.

Conclusão

Entender profundamente a semântica dos verbos HTTP eleva a qualidade do código produzido tanto por desenvolvedores frontend (que criam clientes resilientes, com lógicas de retry em métodos idempotentes) quanto backend (que provisionam infraestruturas escaláveis e conformes com os padrões da web).

Implementar a semântica correta reduz o acoplamento, simplifica o debug e garante que o contrato de integração seja óbvio para qualquer desenvolvedor que chegue no projeto.